

Spring Web on Servlet Stack



Introduction

Servlet-stack web applications

- Built on the Servlet API and deployed to Servlet containers.
- Includes:
 - Spring MVC,
 - View Technologies,
 - CORS (Cross-Origin Resource Sharing) Support,
 - WebSocket Support.

Reactive-stack web applications

- *Spring WebFlux*

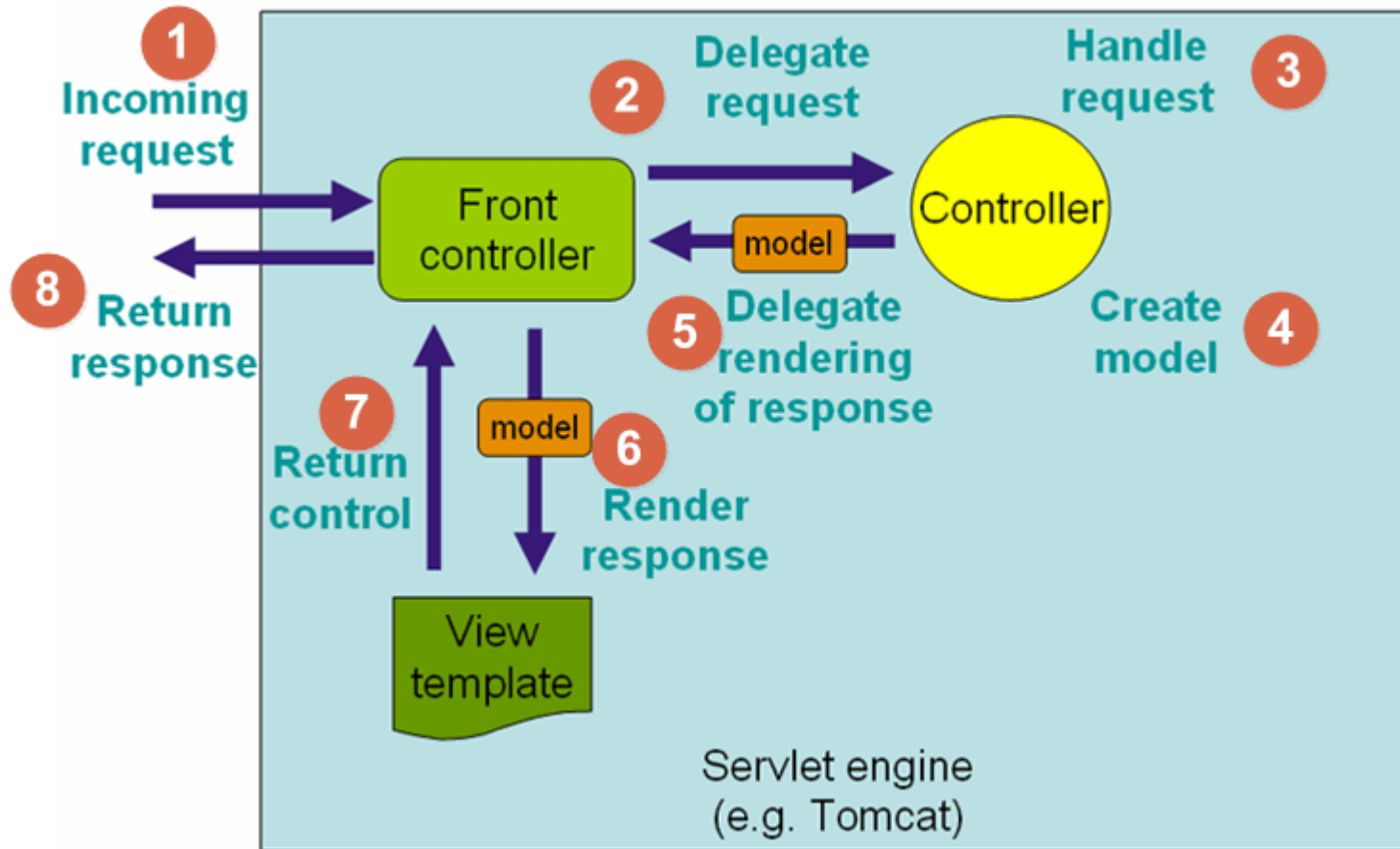
Spring Web MVC Framework

Referred to as Spring MVC, (as many other web frameworks,) is designed around the **front controller** pattern where a central Servlet, (called **DispatcherServlet**), provides a shared algorithm for request processing, while actual work is performed by configurable delegate components.

The **DispatcherServlet**, (as any Servlet,) needs to be declared and mapped according to the Servlet specification by using Java configuration or in [web.xml](#).

In turn, the **DispatcherServlet** uses Spring configuration to discover the delegate components it needs for request mapping, view resolution, exception handling, and other tasks.

The request processing workflow in Spring Web MVC



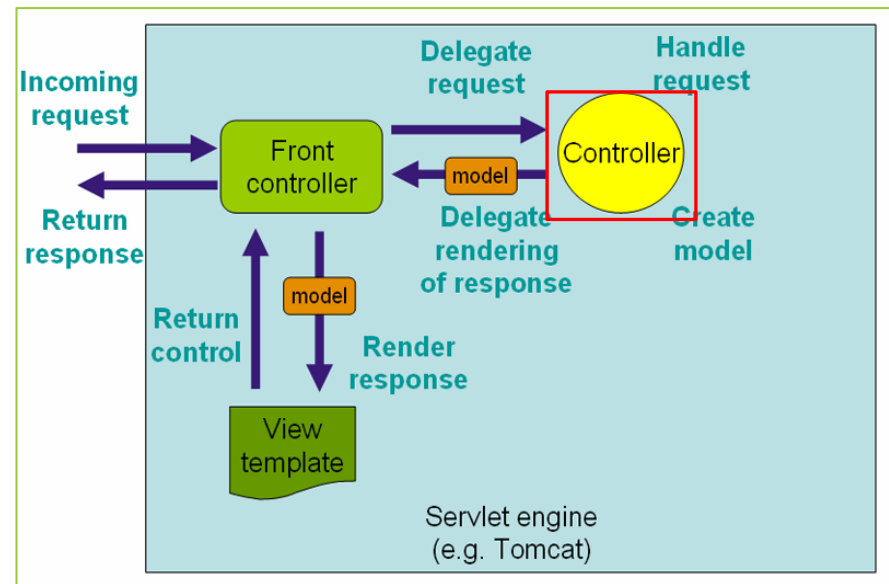
Source: <https://docs.spring.io/>

The Controller

Controllers provide access to the application behavior that you typically define through a service interface.

Controllers interpret user input and transform it into a model that is represented to the user by the view.

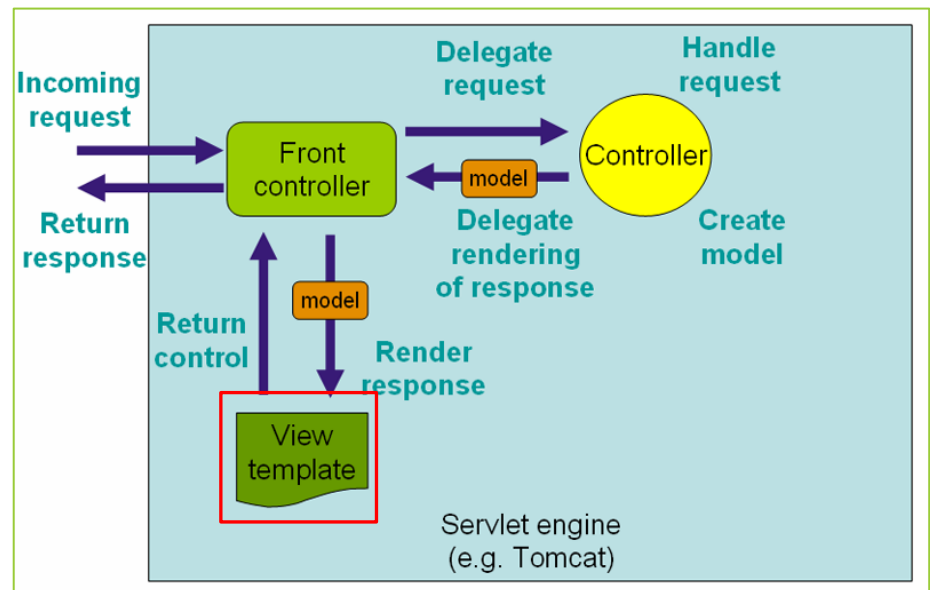
- Spring implements a controller in a very abstract way, which enables you to create a wide variety of controllers.



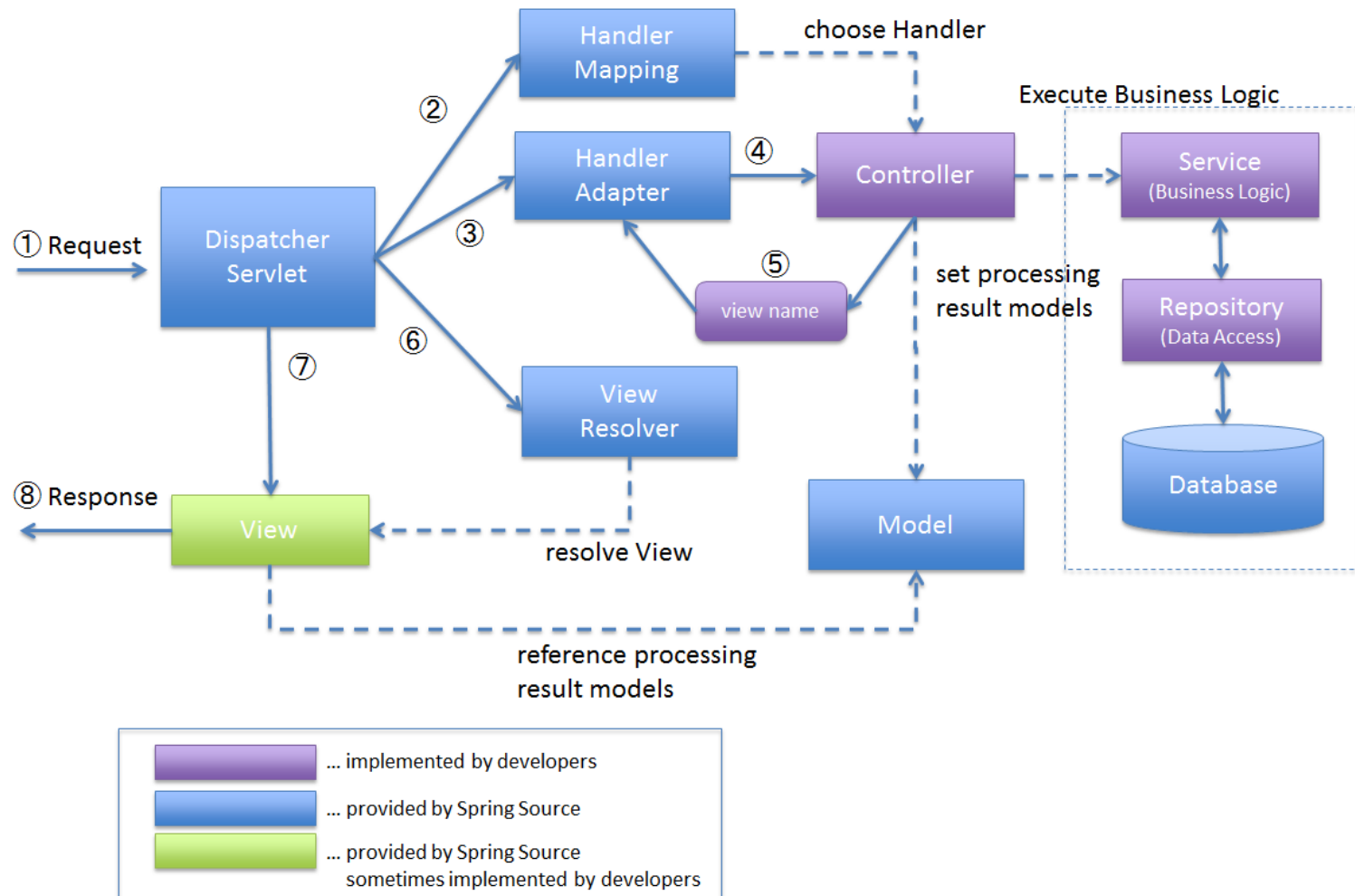
The View

When the Controller completes the request's processing, the Front Controller selects a relevant **view** to render.

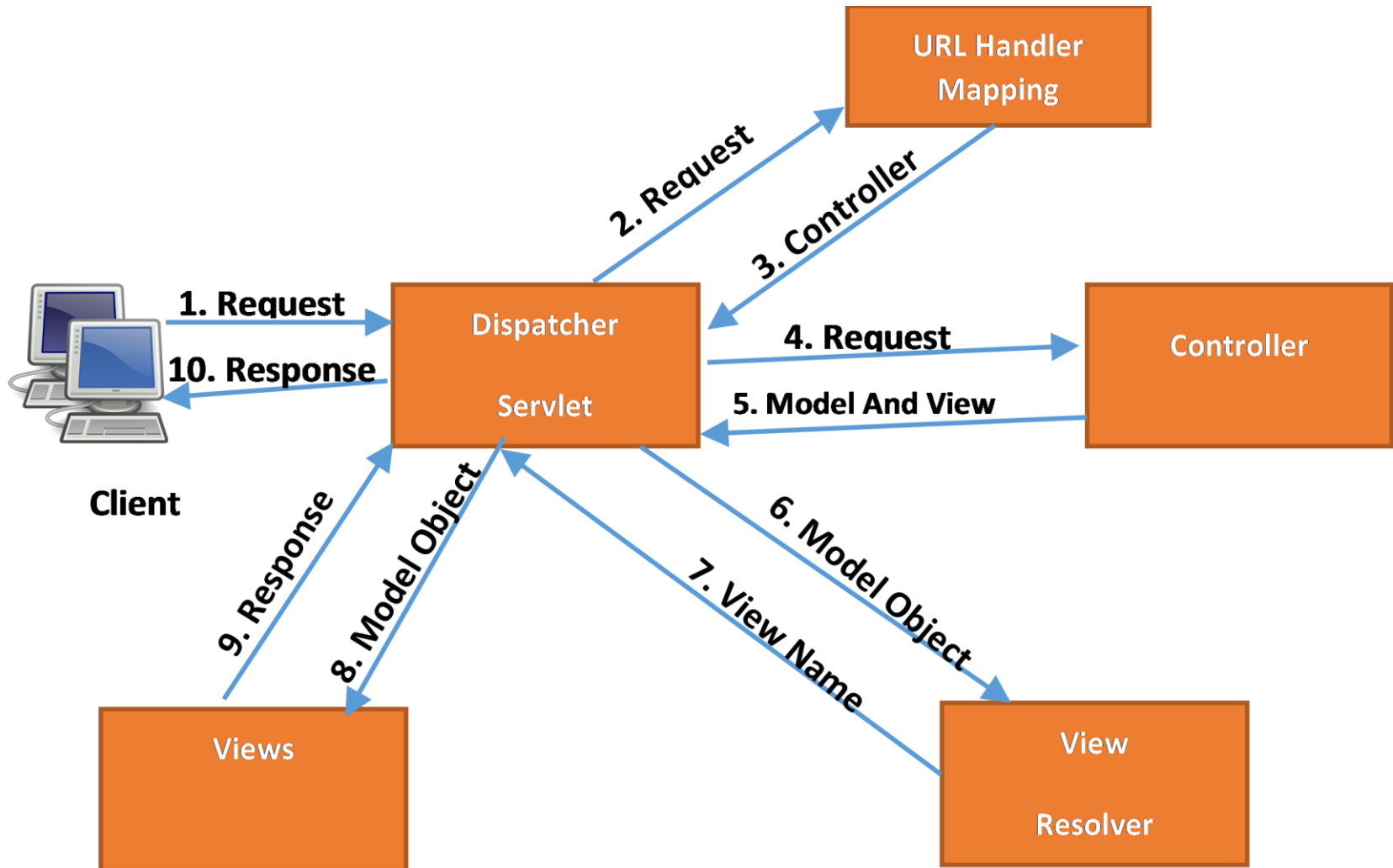
The Front Controller also pass the model(s) to the rendered view on the browser.



The Spring MVC Processing Sequence



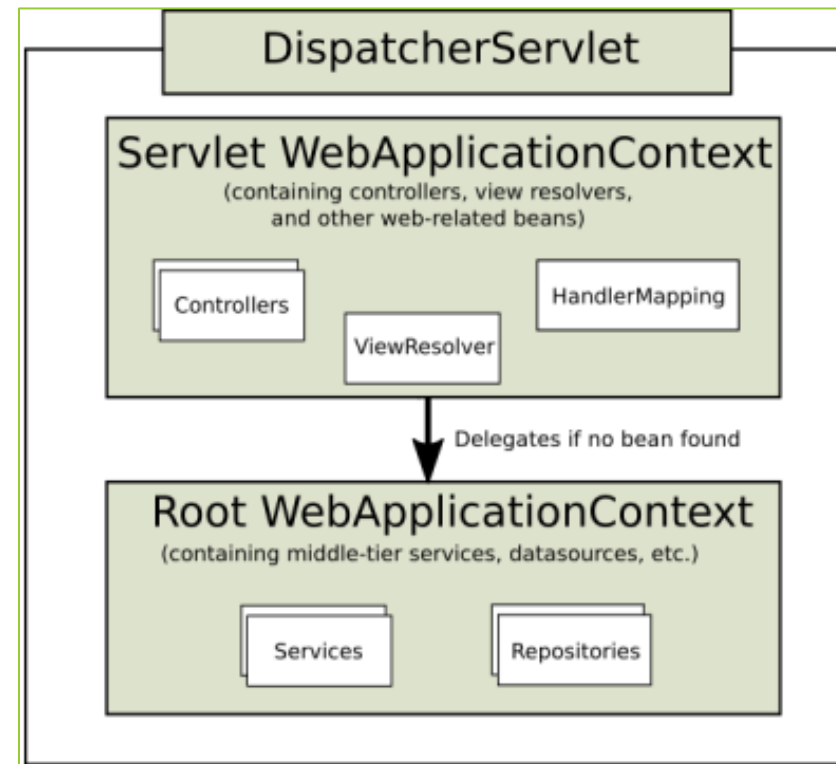
Spring MVC request flow



Context hierarchy in Spring Web MVC

The `ApplicationContext` instances in Spring can be scoped.

- In the Web MVC framework, each `DispatcherServlet` has its own `WebApplicationContext`, which inherits all the beans already defined in the root `WebApplicationContext`.
- These inherited beans can be overridden in the servlet-specific scope, and you can define new scope-specific beans local to a given Servlet instance.





The context package also provides the following functionality:

- Access to messages in i18n-style, through the `MessageSource` interface.
- Access to resources, such as URLs and files, through the `ResourceLoader` interface.
- Event publication to beans implementing the `ApplicationListener` interface, through the use of the `ApplicationEventPublisher` interface.
- Loading of multiple (hierarchical) contexts, allowing each to be focused on one particular layer, such as the web layer of an application, through the `HierarchicalBeanFactory` interface.



Spring Web on Servlet Stack with Springboot



Spring Boot

spring MVC with

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

Features

- Create stand-alone Spring applications
- Embed Tomcat, Jetty or Undertow directly (no need to deploy WAR files)
- Provide opinionated 'starter' dependencies to simplify your build configuration
- Automatically configure Spring and 3rd party libraries whenever possible
- Provide production-ready features such as metrics, health checks, and externalized configuration
- Absolutely no code generation and no requirement for XML configuration

Sample package structure

*Change default Tomcat
server*

- ▼ main
 - ▼ java
 - ▼ fit.se.springdatathymeleafproduct
 - > config
 - > controller
 - > entity
 - > enums
 - > repository
 - > service
 - © ServletInitializer
 - © SpringdataThymeleafProductApplication
- ▼ resources
 - ▼ static.uploads
 - book.jpg
 - ▼ templates
 - category
 - ▼ product

Added dependencies:

- × Spring Boot DevTools
- × Spring Web
- × Thymeleaf
- × Spring Security
- × MariaDB Driver
- × Spring Data JPA

Spring MVC Annotations

`@Controller`, `@RestController` components use annotations to express request mappings, request input, exception handling, and more.

`@RestController` is a composed annotation that is itself meta-annotated.

`@Controller` and `@ResponseBody` to indicate a controller whose every method inherits the type-level `@ResponseBody` annotation and, therefore, writes directly to the response body view resolution and rendering with an HTML template

Spring MVC Annotations (cont.)

@Controller

- Stereotype used as “Controller” of the MVC
- Scanned Request mappings

@RequestMapping

- Used for mapping web requests to specific handler classes and/or handler methods.

name	Assign a name to this mapping.
value	The primary mapping expressed by this annotation.
method	The HTTP request methods to map to, narrowing the primary mapping: GET, POST, HEAD, OPTIONS, PUT, PATCH, DELETE, TRACE.
headers	The headers of the mapped request, narrowing the primary mapping.
consumes	The consumable media types of the mapped request, narrowing the primary mapping.
produces	The producible media types of the mapped request, narrowing the primary mapping.
path	In a Servlet environment only: the path mapping URIs
params	The parameters of the mapped request, narrowing the primary mapping.

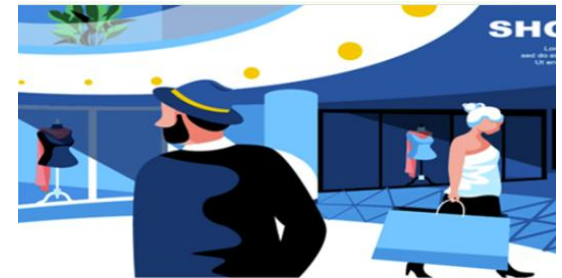

```

@Controller new *
@RequestMapping(🌐"/home")
public class HomeController {

    public HomeController() { new *
        super();
    }

    @GetMapping(value={🌐"",🌐"/trangchu",🌐"/index"}) new *
    public String HomePage(Model model) {
        LocalDate date = LocalDate.now();
        String mess ="Welcome Thymeleaf";
        model.addAttribute( attributeName: "message", mess);
        model.addAttribute( attributeName: "date", date.toString());
        return "home";
    }
}

```



Welcome Thymeleaf

Welcome to shopping!

Today is 2025-10-08
Please select an option

1. [Product List](#)
2. [Order List](#)
3. [Customer List](#)

Copy right ©

Spring MVC Annotations (cont.)

@RequestMapping cont.

- All requests start from “/**home**” will be routed to this controller.
- The **Homepage()** method is accessed using HTTP GET method (default)
- Three URLs:
 - <http://localhost:8085/home>
 - <http://localhost:8085/home/trangchu>
 - <http://localhost:8080/home/index>

are routed to the same “home.html” page.

home.html

```
<body>

<div th:replace="~{navigation :: menu-nav}"></div>

<div class="container-fluid">
  <h1 th:text="${message}">mess</h1>
  <h2 th:text="#{home.welcome}">welcome</h2>
  ⚡ Today is <span th:text="${date}"></span>
  | <p>Please select an option</p>
  <ol>
    <li><a th:href="@{/product}">Product List</a></li>
    <li><a th:href="@{/order}">Order List</a></li>
    <li><a th:href="@{/customer}">Customer List</a></li>

  </ol>
</div>
<div th:insert="~{footer :: footercopy}">
</div>
```

Spring MVC Annotations (cont.)

Since Spring MVC

- `@GetMapping`,
- `@PostMapping`,
- `@PutMapping`,
- `@DeleteMapping`,

equivalently for `@RequestMapping` with specific HTTP methods.

URI patterns

@RequestMapping methods can be mapped using URL patterns. There are two alternatives:

- **PathPattern** — a pre-parsed pattern matched against the URL path also pre-parsed as PathContainer. Designed for web use, this solution deals effectively with encoding and path parameters, and matches efficiently.

```

@Controller new *
@RequestMapping(🌐✓"/customer")
public class CustomerController {

    @Autowired
    private CustomerService customerService;

    @Autowired
    private OrderService orderService;

    @GetMapping() 🌐✓ new *
> public String getAllCustomer(Model model) {...}

    @GetMapping(🌐✓"/{id}/orders") new *
> public String viewOrders(@PathVariable Integer id, Model model) {...}

    @GetMapping(🌐✓"/order/{orderId}") new *
> public String viewOrderDetails(@PathVariable Integer orderId, Model model) {...}
}

```

Spring MVC Annotations (cont.)

@RequestParam

- Annotates that a method **parameter** should be bound to a web request parameter
- Can be passed through the URL query parameters.

Attributes of @RequestParam annotation

defaultValue	The default value to use as a fallback when the request parameter is not provided or has an empty value.
Name	The name of the request parameter to bind to.
Value	It is alias for name attribute.
required	Whether the parameter is required.

```

// Handle update
@PostMapping("/update") new *
public String updateProduct(@ModelAttribute("product") Product product,
                           @RequestParam("fileImage") MultipartFile file) throws IOException {

    // Handle new image upload (optional)
    if (!file.isEmpty()) {
        String fileName = file.getOriginalFilename();
        String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();
        Path uploadDir = Paths.get("uploads/");
        Path uploadDir = Paths.get(first: "src/main/resources/static/uploads/");

        Files.createDirectories(uploadDir);
        file.transferTo(uploadDir.resolve(fileName));
        product.setImage(fileName);
    } else {
        // keep existing image
        Product existing = productService.findById(product.getId()).orElse(other: null);
        if (existing != null) {
            product.setImage(existing.getImage());
        }
    }

    productService.save(product);
    return "redirect:/product";
}

```


Spring MVC Annotations (cont.)

@ModelAttribute

- On a method argument in @RequestMapping methods to create or access an Object from the model and to bind it to the request through a WebDataBinder.
- As a method-level annotation in @Controller or @ControllerAdvice classes that help to initialize the model prior to any @RequestMapping method invocation.
- On a @RequestMapping method to mark its return value is a model attribute

Spring MVC Annotations (cont.)

```
@GetMapping("/addUser")
public ModelAndView addUser(@ModelAttribute("sampleUser") User user){
    //add user to the data source; check if success or not,...

    ModelAndView model=new ModelAndView();
    model.setViewName("listUser");
    return model;
}
```

```
@ModelAttribute
public Account addAccount(@RequestParam String number) {
    return accountRepository.findAccount(number);
}

@GetMapping("/accounts/{id}")
@ModelAttribute("myAccount")
public Account handle() {
    // ...
    return account;
}
```

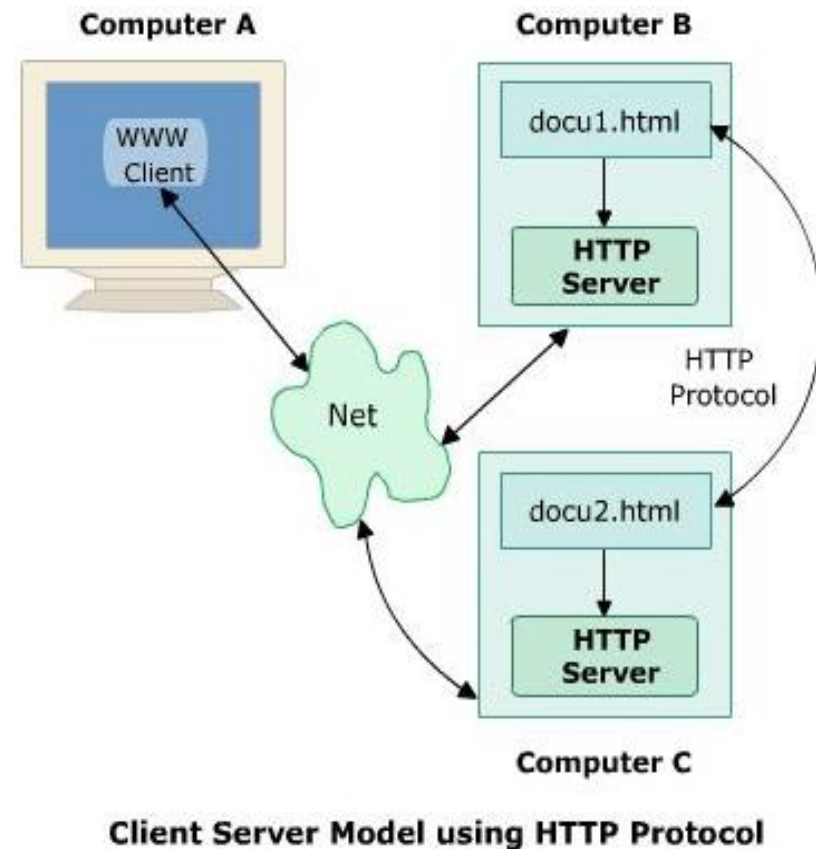


Session Management in Spring MVC

Session Tracking

Protocol

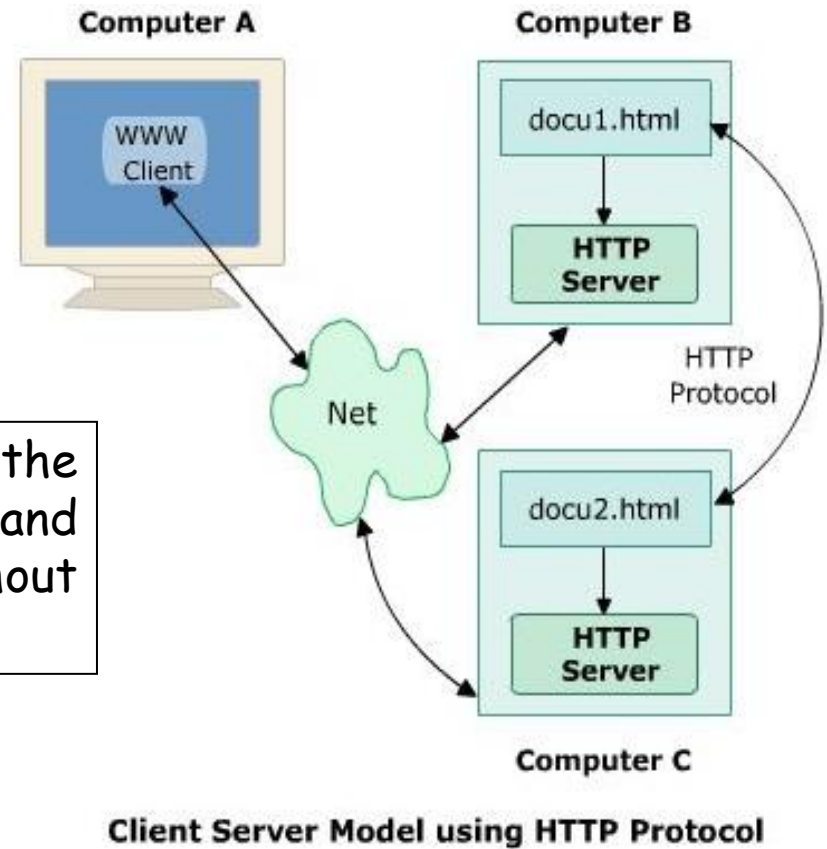
- Is a set of rules, which governs the syntax, semantics and synchronization of communication
- Stateless Protocol: not tracked
- HTTP Protocol
 - Client – server Model
 - Request – response
 - **Stateless** Protocol



Session Tracking

we want to keep data from user,
what will we do?

The session tracking mechanism serves the purpose tracking the client identity and other state information required throughout the session



Session Tracking

Traditional techniques

URL rewriting

Hidden form field

Cookies

- Is a small piece of information sent by the web server to the client to keep track of users.
- Cookie has values in the form of key-value pairs



Concept of Cookie

Session management

Session management is the process of tracking user interactions within a web application.

It allows applications to maintain state across multiple requests and responses, which is essential for implementing features like user authentication, shopping carts, and personalized user experiences.

Spring Session has the simple goal of free up session management from the limitations of the HTTP session stored in the server.

Session management

Here are some common use cases for Spring Session:

- Distributed web applications: If you have a web application distributed across multiple servers, managing user sessions can be challenging. Spring Session can help by storing the session data in a shared database, allowing all servers to access and update session data.

```
<!-- https://mvnrepository.com/artifact/org.springframework.session/spring-session-core -->
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-core</artifactId>
  <version>3.5.2</version>
</dependency>
```


Session management

- Session scalability: In a large web application with many concurrent users, storing sessions in memory on the server can lead to scalability issues. Spring Session allows you to store session data in a persistent store, improving scalability and reducing the risk of out-of-memory errors.
- Session backup and recovery: Storing session data in a persistent store can also provide a mechanism for backing up and recovering session data in case of server failure or downtime.

Session management

In Spring Boot, session management revolves around the `HttpSession` interface, which provides a way to store and retrieve user-specific information during the course of a session.

Spring Boot's session management builds on top of the underlying servlet container's session handling mechanisms.

- **`HttpServletRequest`**
- **`HttpSession`**
- **`@SessionAttribute` annotation (***)**

Session management

HttpServletRequest

```
@Controller new *
public class SessionController {
    @GetMapping("/set") new *
    public String setSession(HttpServletRequest request, Model model) {
        HttpSession session = request.getSession();
        session.setAttribute(s: "username", o: "Mai Hoang Lan");
        model.addAttribute(attributeName: "msg", attributeValue: "Session set thành công!");
        return "session";
    }

    @GetMapping("/get") new *
    public String getSession(HttpSession session, Model model) {
        String username = (String) session.getAttribute(s: "username");
        model.addAttribute(attributeName: "msg", attributeValue: "Giá trị session: " + username);
        return "session";
    }
}
```

Session management

HttpSession

```
@Controller new *
@RequestMapping("/demosession")
public class UsingHttpSessionController {

    @GetMapping("/set-v1") new *
    public String setV1(HttpSession session, Model model) {
        session.setAttribute(s: "name", new Task( taskname: "Task Example"));
        model.addAttribute( attributeName: "message", attributeValue: "Task Example cre
        return "session-view";
    }

    @GetMapping("/get-v1") new *
    public String getV1(HttpSession session, Model model) {
        Task task = (Task) session.getAttribute(s: "name");
        if(task != null) {
            model.addAttribute( attributeName: "task", task);
            model.addAttribute( attributeName: "message", attributeValue: "Getting the
        }
    }
}
```

Session management

@SessionAttribute annotation

- Replacing of using the HttpSession object (since v4.3).
- Retrieves the existing attribute from session to a handler method parameter.

```
@Controller new *
@SessionAttributes("user")
public class UserController {
    @Autowired
    private UserService userService;
    @ModelAttribute("user") new *
    public User user() {
        return new User();
    }
    @GetMapping("/") new *
    public String loginForm(Model model) {
        return "login";
    }
    @PostMapping("/login") new *
    public String login(@ModelAttribute("user") User user, Model model) {
        User existsuser = userService.findByUsername(user.getUsername());
        if (existsuser != null && existsuser.getPassword().equals(user.getPassword())) {
            model.addAttribute(attributeName: "user", user);
            return "profile";
        }
        return "login";
    }
}
```



Session management

Session Store in database - JDBC

Spring Session provides a layer of abstraction between the application and the session management. It allows the session data to be stored in various persistent stores, such as relational databases, NoSQL databases, and others.

With Spring Session, you can use the same API to manage sessions, regardless of the persistent store used. This makes it easier to switch between stores without changing the application code.

Spring Session also provides features such as session expiry and cross-context communication between different web applications.

1. Session JDBC dependencies

```
<dependency>  
  <groupId>org.springframework.session</groupId>  
  <artifactId>spring-session-jdbc</artifactId>  
</dependency>
```

1. Config application.properties

= Database Configuration

```
spring.datasource.url=jdbc:mariadb://localhost:3306/dienthoaidb  
spring.datasource.username=root  
spring.datasource.password=root  
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
```

= session jdbc

```
spring.session.jdbc.initialize-schema=always
```


Spring MVC Return Types

ModelAndView Object

Model Object

Map for exposing model

View Object

String which is the view name

Void (if method wrote the response content directly)